Universidad Nacional de Colombia - Sede Bogotá
Ingeniería de Software II
Laboratorio Patrones de Diseño
Sebastian Clavijo Tocasuchyl

A continuación, se detalla el funcionamiento lógico de los patrones implementados en la guía del laboratorio y cómo resuelven problemas específicos de diseño:

1. Patrones Creacionales (Cómo se crean los objetos)

- **Builder**: Soluciona el problema de crear objetos con múltiples configuraciones u opcionales. La clase `CoffeeBuilder` asume la responsabilidad de ensamblar el objeto `Coffee` paso a paso (añadir leche, configurar tamaño), devolviendo el objeto final solo cuando se ejecuta el método `build()`.

- **Prototype**: Es útil cuando crear un objeto desde cero requiere muchos recursos o pasos. La clase `PromoBanner` implementa un método `clone` usando `copy.deepcopy` para duplicar exactamente el estado del objeto "prototipo", permitiendo alterar únicamente los detalles necesarios en la nueva copia.

- **Singleton**: Garantiza que, durante todo el ciclo de vida de la aplicación, solo exista una única instancia de una clase. Al sobrescribir el método de instanciación `__new__` en `AppConfig`, el código intercepta la creación de nuevos objetos y retorna siempre la misma dirección en memoria previamente almacenada.

- **Factory Method**: Delega la responsabilidad de elegir qué objeto instanciar a un método de fabricación. El cliente solo pasa un parámetro de texto (ej. "bar", "fizzy") a `WonkaFactory`, y la fábrica contiene la lógica interna para retornar la clase hija correcta (`ChocolateBar`, `FizzyLiftingDrink`).

```
--- 1.1 Builder ---
Coffee (size=grande, milk=almendra, sugar=2, extras=['canela'])
Coffee (size=pequeño, milk=None, sugar=0, extras=[])

--- 1.2 Prototype ---
[Promo de la semana] Café + croissant (20% off)
[Lunes de energía] Café + croissant (20% off)
[Promo de la semana] Café + croissant (30% off)

--- 1.3 Singleton ---
Config1: postgres://prod
Config2: postgres://prod
¿Son el mismo objeto?: True

--- 1.4 Factory Method ---
Preparando una barra de chocolate Wonka
Preparando un Fizzy Lifting Drink (bebida burbujeante)
Preparando un Everlasting Gobstopper (caramelo eterno)
```

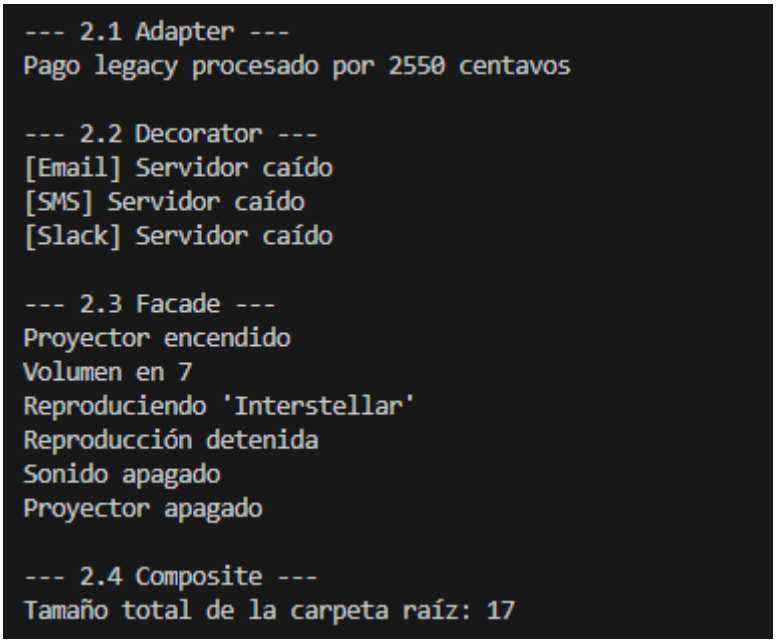
2. Patrones Estructurales (Cómo se relacionan clases y objetos)

- **Adapter**: Actúa como un traductor entre dos sistemas incompatibles. En el código, `LegacyPaymentAdapter` cumple con la interfaz moderna (método `pay` en dólares) pero por dentro multiplica por 100 para consumir el método antiguo `make_payment` (en centavos) del sistema *legacy*.

- **Decorator**: Permite apilar o envolver funcionalidades sobre un objeto original en tiempo de ejecución. Pasando un notificador base a `SMSDecorator`, y este a su vez a `SlackDecorator`, se logra que un solo llamado al método `send()` ejecute en cascada todas las capas de notificación apiladas.

- **Facade**: Simplifica la interacción ocultando múltiples componentes de un subsistema. El usuario no tiene que encender el proyector, el sonido y el reproductor de forma independiente; en su lugar, `HomeTheaterFacade` orquesta internamente a las tres clases en un solo método público `ver_pelicula()`.

- **Composite**: Unifica el trato entre elementos simples y jerarquías. Al hacer que un archivo (`File`) y un directorio (`Folder`) hereden de `FileSystemItem`, ambos tienen un método `get_size()`. Si le preguntas el tamaño a una carpeta, esta sabe cómo sumar el tamaño de sus propios archivos y de cualquier subcarpeta recursivamente.



3. Patrones de Comportamiento (Cómo interactúan y se comunican)

- **Strategy**: Aísla lógicas intercambiables para evitar condicionales masivos. El carrito de compras (`Cart`) no sabe cómo calcular descuentos, solo ejecuta `strategy.apply()`. Esto permite inyectar en tiempo real distintos objetos de descuento (`StudentDiscount`, `VIPDiscount`) según se requiera.

- **Observer**: Crea un sistema de suscripción descentralizado. El canal de video (`Channel`) mantiene una lista genérica de objetos `Subscriber`. Al publicar, itera sobre esa lista y ejecuta `update()`, logrando que todos reaccionen sin que el canal tenga que saber si son correos o notificaciones móviles.
- **Command**: Transforma una instrucción temporal en un objeto persistente. En lugar de simplemente encender la luz, el sistema crea un objeto `TurnOnCommand` que posee un método `execute()` y un método `undo()`. Esto permite guardar el objeto en un historial (`RemoteControl`) para revertir operaciones pasadas.

- **Template Method**: Fija la receta o el esqueleto estricto de un algoritmo, pero permite que las clases hijas rellenen los huecos. `Report.generate` establece firmemente que un reporte lleva cabecera, cuerpo y pie. Las clases `TextReport` o `HTMLReport` heredan esa estructura y solo se preocupan por codificar el formato visual de cada parte.

- **State**: Permite a una máquina de estados alterar su comportamiento a medida que transiciona. El pedido (`Order`) no usa sentencias `if (estado == 'Pendiente')`. En su lugar, delega las acciones `pay()` o `ship()` a un objeto de estado dinámico (`Pending`, `Paid`), el cual controla si la acción es válida y avanza al siguiente paso.

